Competitive Programming

Section 45 Editorial: Flows and Matchings

Competitive Programming Course

2026-10-28

Table of contents

1 How to use this editorial **5**
- 1.1 Engine conventions . . . 5

2 A. Max Flow **6**
- 2.1 Model, trace, and algorithm . . . 6
- 2.2 Correctness . . . 6
- 2.3 Complexity and traps . . . 6
- 2.4 Full C++ solution . . . 6
- 2.5 Full Python solution . . . 9

3 B. Bipartite Matching **12**
- 3.1 Model, trace, and algorithm . . . 12
- 3.2 Correctness . . . 12
- 3.3 Complexity and traps . . . 12
- 3.4 Full C++ solution . . . 12
- 3.5 Full Python solution . . . 14

4 C. Min Cut Reachable Side **16**
- 4.1 Model, trace, and algorithm . . . 16
- 4.2 Correctness . . . 16
- 4.3 Complexity and traps . . . 16
- 4.4 Full C++ solution . . . 16
- 4.5 Full Python solution . . . 19

5 D. Project Selection **22**
- 5.1 Model and sample trace . . . 22
- 5.2 Algorithm and correctness . . . 22
- 5.3 Complexity and traps . . . 22
- 5.4 Full C++ solution . . . 22
- 5.5 Full Python solution . . . 24

6 E. Separated Light Marks **27**
- 6.1 Restatement and key observation . . . 27
- 6.2 Trace and algorithm . . . 27
- 6.3 Correctness proof . . . 27
- 6.4 Bounds, implementation notes, and tests . . . 27
- 6.5 Full C++ solution . . . 28
- 6.6 Full Python solution . . . 30

7 F. Crate Routing **32**
- 7.1 Restatement and model . . . 32
- 7.2 Sample trace and algorithm . . . 32
- 7.3 Correctness proof . . . 32
- 7.4 Bounds, implementation notes, and tests . . . 33
- 7.5 Full C++ solution . . . 33

7.6 Full Python solution 36

8 G. Silent Rehearsal **39**
8.1 Restatement and threshold model 39
8.2 Sample trace and algorithm 39
8.3 Correctness proof 39
8.4 Bounds, implementation notes, and tests 39
8.5 Full C++ solution 40
8.6 Full Python solution 42

9 H. Emergency Corridors **44**
9.1 Restatement and modeling obstacle 44
9.2 Trace and algorithm 44
9.3 Correctness proof 44
9.4 Bounds, implementation notes, and tests 45
9.5 Full C++ solution 45
9.6 Full Python solution 48

10 I. Directional Relay Modes **51**
10.1 Restatement and cut convention 51
10.2 Truth table and sample 51
10.3 Correctness proof 51
10.4 Bounds, implementation notes, and tests 52
10.5 Full C++ solution 52
10.6 Full Python solution 55

11 J. Exhibit Pacts **58**
11.1 Restatement and why one auxiliary vertex helps 58
11.2 What each auxiliary choice means 58
11.3 Sample trace and algorithm 58
11.4 Correctness proof 59
11.5 Bounds, implementation notes, and tests 59
11.6 Full C++ solution 59
11.7 Full Python solution 62

12 K. Quota Manifests **66**
12.1 Restatement and bounded network 66
12.2 Convert lower bounds to ordinary flow 66
12.3 Sample trace and correctness 66
12.4 Bounds, implementation notes, and tests 67
12.5 Full C++ solution 67
12.6 Full Python solution 70

13 L. Calendar Reinforcement **74**
13.1 Restatement and monotonicity 74
13.2 Compress equivalent days 74
13.3 Why aggregation does not invent an impossible schedule 74
13.4 Sample trace, algorithm, and correctness 75
13.5 Bounds, implementation notes, and tests 75
13.6 Full C++ solution 75
13.7 Full Python solution 79

14 M. Resilient Reservations **82**
14.1 Restatement and the right adversarial condition . . . 82
14.2 Find minimum slack without enumerating groups . . . 82
14.3 Sample trace and algorithm . . . 82
14.4 Bounds, implementation notes, and tests . . . 83
14.5 Full C++ solution . . . 83
14.6 Full Python solution . . . 86

15 N. Signal Concentration **89**
15.1 Restatement and why total reward is insufficient . . . 89
15.2 Turn a ratio threshold into an additive objective . . . 89
15.3 Exact binary search and extraction . . . 89
15.4 Why the extracted ratio is exactly optimal . . . 90
15.5 Bounds, implementation notes, and tests . . . 90
15.6 Full C++ solution . . . 90
15.7 Full Python solution . . . 94

16 Testing and what to carry forward **97**

1 How to use this editorial

Attempt a problem from its standalone statement before reading its chapter. A-C practice the algorithms, D is the original blind exercise, and E-N are an increasing modeling progression. Difficulty estimates in the section index are teaching targets, not official ratings.

For every model below, identify what a vertex represents, what one unit or one side of a cut means, and prove the correspondence in both directions. Full C++ and Python solutions follow each explanation. The three introductory references and the extended references use the reusable iterative searches from the lesson; they do not depend on Python's recursion limit.

1.1 Engine conventions

`Dinic.add_edge(u,v,c)` creates a forward residual entry of capacity `c` and a reverse entry of capacity zero. IDs are paired by `edge ^ 1`. `max_flow(s,t)` sends additional flow and mutates the residual capacities; `reachable(s)` then returns the positive-residual source side. Rebuild the network for independent binary-search candidates. With `V` vertices and `E` input edges, use the general `O(V^2 E)` time and `O(V+E)` space bounds. For an edgeless network, the final BFS costs `O(V)`.

The direct matching class stores current partners in `left` and `right`, using `-1` for free vertices. Its iterative alternating BFS costs `O(L*(E+L+R))` in the stated implementation, including initialization of search arrays. Space is `O(E+L+R)`. Both classes use independent 0-based IDs; convert statement IDs on input. C++ capacities and accumulated answers are `long long`; Python integers are unbounded.

2 A. Max Flow

2.1 Model, trace, and algorithm

Each input edge already is a capacity constraint. Build the same network and run Dinic from vertex 1 to vertex `n`. BFS assigns residual distances; within a phase, send bottlenecks only along positive arrows advancing one level. Update the reverse permission with every push, and repeat full BFS until the sink becomes unreachable.

On the sample, phase 1 sends two units on 1-2-4 and two on 1-3-4. The next BFS permits 1-2-3-4, adding one more unit. The answer is 5, not 4: a blocked level graph ends only the phase.

2.2 Correctness

Each residual augmentation preserves capacity bounds and conservation while increasing net source-to-sink flow by its bottleneck. At termination, no positive residual arrow leaves the source-reachable set. Outgoing input edges are full and incoming input edges carry zero flow, so current flow equals that cut's capacity. Every feasible flow is bounded by this capacity; the current flow is therefore maximum.

2.3 Complexity and traps

`V=n`, `E=m`: `O(V^2 E)` time and `O(V+E)` space. Use 64-bit totals. Parallel and opposite input edges keep separate residual pairs; never merge an input edge with another edge's cancellation permission. Test zero edges, zero capacities, a long chain, rerouting, and source capacities whose sum exceeds 32 bits.

2.4 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
```

```
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
```

```
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
```

```
                }
            }
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    Dinic flow(n);
    while (m--) {
        int u, v; long long capacity;
        cin >> u >> v >> capacity;
        flow.add_edge(u - 1, v - 1, capacity);
    }
    cout << flow.max_flow(0, n - 1) << '\n';
}
```

2.5 Full Python solution

```
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge
```

```
    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
```

```
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen


def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n, m = next(it), next(it)
    flow = Dinic(n)
    for _ in range(m):
        u, v, capacity = next(it), next(it), next(it)
        flow.add_edge(u - 1, v - 1, capacity)
    print(flow.max_flow(0, n - 1))


if __name__ == '__main__':
    main()
```

3 B. Bipartite Matching

3.1 Model, trace, and algorithm

The two sides and allowed pairs are already given. For each free left root, explore unused pairs to right vertices and occupied pairs back to their owners. Reaching a free right vertex supplies an alternating path: flip it and gain one pair. If exploration fails, leave the matching unchanged.

On the sample, selecting 1-1, 2-2, and 3-3 accommodates all three left vertices. To test reassignment, instead use L1 adjacent to R1/R2 and L2 adjacent only to R1: the second search moves L1 to R2 and gives R1 to L2.

3.2 Correctness

A flip adds one more unused pair than the matched pairs it removes, and preserves endpoint uniqueness. A larger matching compared with the current one contains an alternating path starting and ending at free vertices. The search explores every continuation from its root. One pass over roots suffices: successful roots stay matched; a failed search discovers a closed region with no free slot, untouched by later successful augmentations. Thus no augmenting path remains at the end, proving maximal size.

3.3 Complexity and traps

The included BFS matcher takes `O(n*(e+n+m))` time and `O(e+n+m)` space. Reset search state per root while retaining partners. Do not confuse `previous[v]`, the potential new owner during exploration, with `right[v]`, the current owner. Test Hall-deficient groups, isolated vertices, unequal side sizes, and long alternating paths.

3.4 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <cassert>
#include <queue>
#include <utility>
#include <vector>

struct BipartiteMatching {
    std::vector<std::vector<int>> adj;
```

```
    std::vector<int> left, right;

    BipartiteMatching(int left_count, int right_count)
        : adj(left_count), left(left_count, -1), right(right_count, -1) {}

    void add_edge(int u, int v) {
        assert(0 <= u && u < static_cast<int>(left.size()));
        assert(0 <= v && v < static_cast<int>(right.size()));
        adj[u].push_back(v);
    }

    bool augment(int root) {
        std::vector<int> previous(right.size(), -1);
        std::vector<bool> seen(left.size(), false);
        std::queue<int> queue;
        seen[root] = true;
        queue.push(root);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int v : adj[u]) {
                if (previous[v] != -1) continue;
                previous[v] = u;
                int owner = right[v];
                if (owner == -1) {
                    while (v != -1) {
                        u = previous[v];
                        int old = left[u];
                        left[u] = v;
                        right[v] = u;
                        v = old;
                    }
                    return true;
                }
                if (!seen[owner]) {
                    seen[owner] = true;
                    queue.push(owner);
                }
            }
        }
        return false;
    }

    std::vector<std::pair<int, int>> maximum_matching() {
        for (int root = 0; root < static_cast<int>(left.size()); ++root)
            if (left[root] == -1) augment(root);
        std::vector<std::pair<int, int>> pairs;
        for (int u = 0; u < static_cast<int>(left.size()); ++u)
            if (left[u] != -1) pairs.push_back({u, left[u]});
        return pairs;
    }
```

```cpp
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e;
    if (!(cin >> n >> m >> e)) return 0;
    BipartiteMatching matching(n, m);
    while (e--) {
        int u, v; cin >> u >> v;
        matching.add_edge(u - 1, v - 1);
    }
    cout << matching.maximum_matching().size() << '\n';
}
```

3.5 Full Python solution

```python
import sys
from collections import deque


class BipartiteMatching:
    """Independent 0-based IDs on each side. -1 means unmatched."""

    def __init__(self, left_count, right_count):
        self.adj = [[] for _ in range(left_count)]
        self.left = [-1] * left_count
        self.right = [-1] * right_count

    def add_edge(self, u, v):
        assert 0 <= u < len(self.left) and 0 <= v < len(self.right)
        self.adj[u].append(v)

    def _augment(self, root):
        previous = [-1] * len(self.right)
        seen = [False] * len(self.left)
        seen[root] = True
        queue = deque([root])
        while queue:
            u = queue.popleft()
            for v in self.adj[u]:
                if previous[v] != -1:
                    continue
                previous[v] = u
                owner = self.right[v]
                if owner == -1:
                    # Follow predecessors and flip the entire path.
                    while v != -1:
```

```
                        u = previous[v]
                        old = self.left[u]
                        self.left[u] = v
                        self.right[v] = u
                        v = old
                    return True
                if not seen[owner]:
                    seen[owner] = True
                    queue.append(owner)
        return False

    def maximum_matching(self):
        for root in range(len(self.left)):
            if self.left[root] == -1:
                self._augment(root)
        return [(u, v) for u, v in enumerate(self.left) if v != -1]


def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n, m, e = next(it), next(it), next(it)
    matching = BipartiteMatching(n, m)
    for _ in range(e):
        u, v = next(it), next(it)
        matching.add_edge(u - 1, v - 1)
    print(len(matching.maximum_matching()))


if __name__ == '__main__':
    main()
```

4 C. Min Cut Reachable Side

4.1 Model, trace, and algorithm

Run Dinic on the input network. Then run one BFS from vertex 1 over all positive final residual arrows, including cancellation arrows. Print the flow and reached original vertex IDs in increasing order.

The sample has value 5. Both source edges are saturated, so the reached set is just vertex 1. An original edge with positive capacity may now be unusable; original-graph reachability would return the wrong set.

4.2 Correctness

The final BFS's set `S` contains the source and excludes the sink. An input edge leaving `S` is full; an input edge entering `S` has zero flow, since otherwise its cancellation arrow would let the BFS leave. Consequently, flow value equals the input capacity leaving `S`. This is a minimum cut by max-flow/min-cut, and the printed set is exactly the requested residual reachable set. The final BFS adds `O(n+m)` time to Dinic.

4.3 Complexity and traps

Total time is `O(n^2*m)` with `O(n+m)` memory, plus the final linear BFS. The no-edge case is linear. Test zero flow, zero-capacity input edges, parallel edges, vertices reached only through cancellation, and output order.

4.4 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
```

```
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
```

```
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
```

```
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    Dinic flow(n);
    while (m--) {
        int u, v; long long capacity;
        cin >> u >> v >> capacity;
        flow.add_edge(u - 1, v - 1, capacity);
    }
    cout << flow.max_flow(0, n - 1) << '\n';
    auto seen = flow.reachable(0);
    bool first = true;
    for (int v = 0; v < n; ++v) if (seen[v]) {
        if (!first) cout << ' ';
        first = false;
        cout << v + 1;
    }
    cout << '\n';
}
```

4.5 Full Python solution

```
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
```

```
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0
```

```
    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen


def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n, m = next(it), next(it)
    flow = Dinic(n)
    for _ in range(m):
        u, v, capacity = next(it), next(it), next(it)
        flow.add_edge(u - 1, v - 1, capacity)
    print(flow.max_flow(0, n - 1))
    seen = flow.reachable(0)
    print(' '.join(str(v + 1) for v in range(n) if seen[v]))


if __name__ == '__main__':
    main()
```

5 D. Project Selection

5.1 Model and sample trace

A project selected on the source side contributes its profit. Put a positive profit `w` on `s -> project` and a negative profit `w` on `project -> t` with capacity `-w`. If selecting `a` requires `b`, add `a -> b` with capacity `INF`. Let `P` be the sum of positive profits and use `INF=P+1`.

For profits `[9,-5,7,-3]` and rules 1->2, 3->4, 3->2, selecting all projects earns 8. Its cut pays negative costs `5+3=8`; `P=16`, so `P-cut=8`. Selecting only 1 and 2 earns 4: the cut loses positive profit 7 on project 3 and pays cost 5 on project 2, giving `16-12=4`.

5.2 Algorithm and correctness

```
P = sum of positive profits
add profit edges and implication edges of capacity P+1
answer = P - max_flow(s,t)
```

Any feasible selection gives a cut with capacity `P - selected_profit`: unselected positive projects lose their profit and selected negative ones pay their cost. Conversely, the cut selecting no projects costs `P`, so an optimal cut cannot cross an implication of capacity `P+1`. Its selected projects satisfy every prerequisite, and the same identity applies. Minimizing the cut therefore maximizes the profit. Empty selection, cycles of prerequisites, and zero profits are all handled without special graph preprocessing.

5.3 Complexity and traps

`V=n+2`, `E=n+m`, so time is `O(V^2 E)` and space `O(V+E)`. Use 64-bit profits and `INF`; a fixed small infinity may silently allow a forbidden selection. The original D reference uses recursive Dinic with the local `n<=200` bound; Python sets its recursion limit explicitly. Tests enumerate all small project subsets and include all-negative projects and prerequisites that make an attractive individual project unprofitable.

5.4 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Dinic {
    struct Edge {
        int to, reverse;
        long long capacity;
    };

    vector<vector<Edge>> graph;
    vector<int> level, next_edge;

    explicit Dinic(int n) : graph(n), level(n), next_edge(n) {}

    void add_edge(int from, int to, long long capacity) {
        Edge forward{to, (int)graph[to].size(), capacity};
        Edge backward{from, (int)graph[from].size(), 0};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    bool build_levels(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> queue;
        level[source] = 0;
        queue.push(source);
        while (!queue.empty()) {
            int vertex = queue.front();
            queue.pop();
            for (const Edge& edge : graph[vertex]) {
                if (edge.capacity > 0 && level[edge.to] == -1) {
                    level[edge.to] = level[vertex] + 1;
                    queue.push(edge.to);
                }
            }
        }
        return level[sink] != -1;
    }

    long long send_flow(int vertex, int sink, long long pushed) {
        if (vertex == sink) return pushed;
        for (int& edge_index = next_edge[vertex];
             edge_index < (int)graph[vertex].size(); edge_index++) {
            Edge& edge = graph[vertex][edge_index];
            if (edge.capacity == 0 || level[edge.to] != level[vertex] + 1) continue;
            long long flow = send_flow(edge.to, sink, min(pushed, edge.capacity));
            if (flow == 0) continue;
            edge.capacity -= flow;
            graph[edge.to][edge.reverse].capacity += flow;
            return flow;
        }
```

```
        return 0;
    }

    long long max_flow(int source, int sink) {
        long long total = 0;
        const long long INF = numeric_limits<long long>::max() / 4;
        while (build_levels(source, sink)) {
            fill(next_edge.begin(), next_edge.end(), 0);
            while (long long pushed = send_flow(source, sink, INF)) total += pushed;
        }
        return total;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<long long> profit(n);
    long long positive_total = 0;
    for (long long& value : profit) {
        cin >> value;
        if (value > 0) positive_total += value;
    }

    int source = n;
    int sink = n + 1;
    Dinic flow(n + 2);
    long long inf = positive_total + 1;
    for (int project = 0; project < n; project++) {
        if (profit[project] > 0) flow.add_edge(source, project, profit[project]);
        if (profit[project] < 0) flow.add_edge(project, sink, -profit[project]);
    }
    for (int i = 0; i < m; i++) {
        int project, prerequisite;
        cin >> project >> prerequisite;
        flow.add_edge(project - 1, prerequisite - 1, inf);
    }

    cout << positive_total - flow.max_flow(source, sink) << '\n';
    return 0;
}
```

5.5 Full Python solution

```
import sys
from collections import deque
```

```python
class Dinic:
    def __init__(self, size: int) -> None:
        self.graph: list[list[list[int]]] = [[] for _ in range(size)]
        self.level = [-1] * size
        self.next_edge = [0] * size

    def add_edge(self, start: int, end: int, capacity: int) -> None:
        self.graph[start].append([end, len(self.graph[end]), capacity])
        self.graph[end].append([start, len(self.graph[start]) - 1, 0])

    def build_levels(self, source: int, sink: int) -> bool:
        self.level = [-1] * len(self.graph)
        self.level[source] = 0
        queue = deque([source])
        while queue:
            vertex = queue.popleft()
            for target, _, capacity in self.graph[vertex]:
                if capacity > 0 and self.level[target] == -1:
                    self.level[target] = self.level[vertex] + 1
                    queue.append(target)
        return self.level[sink] != -1

    def send_flow(self, vertex: int, sink: int, pushed: int) -> int:
        if vertex == sink:
            return pushed
        while self.next_edge[vertex] < len(self.graph[vertex]):
            edge = self.graph[vertex][self.next_edge[vertex]]
            target, reverse, capacity = edge
            if capacity > 0 and self.level[target] == self.level[vertex] + 1:
                flow = self.send_flow(target, sink, min(pushed, capacity))
                if flow:
                    edge[2] -= flow
                    self.graph[target][reverse][2] += flow
                    return flow
            self.next_edge[vertex] += 1
        return 0

    def max_flow(self, source: int, sink: int) -> int:
        total = 0
        while self.build_levels(source, sink):
            self.next_edge = [0] * len(self.graph)
            while pushed := self.send_flow(source, sink, 10**30):
                total += pushed
        return total


def main() -> None:
    tokens = list(map(int, sys.stdin.buffer.read().split()))
    if not tokens:
```

```
        return
    n, m = tokens[0], tokens[1]
    profit = tokens[2 : 2 + n]
    pointer = 2 + n
    positive_total = sum(value for value in profit if value > 0)
    source, sink = n, n + 1
    flow = Dinic(n + 2)
    inf = positive_total + 1
    for project, value in enumerate(profit):
        if value > 0:
            flow.add_edge(source, project, value)
        elif value < 0:
            flow.add_edge(project, sink, -value)
    for _ in range(m):
        project, prerequisite = tokens[pointer] - 1, tokens[pointer + 1] - 1
        pointer += 2
        flow.add_edge(project, prerequisite, inf)
    print(positive_total - flow.max_flow(source, sink))


if __name__ == "__main__":
    sys.setrecursionlimit(300_000)
    main()
```

6 E. Separated Light Marks

6.1 Restatement and key observation

Choose floor cells so no two share an uninterrupted horizontal or vertical run. A partition splits a row or column into different runs, so whole-grid row IDs and column IDs are too coarse.

Give each maximal horizontal floor run one left vertex, and each maximal vertical floor run one right vertex. A floor cell is the intersection of exactly one run of each kind; add their compatibility edge. Selecting that edge means putting a mark on that cell.

6.2 Trace and algorithm

In the sample, every floor cell is an isolated run in both directions, so four disjoint edges can all be selected. More instructively, for the two rows `...` and `.#.`, there are three horizontal runs and three vertical runs. Put a mark in the top middle and in both bottom corners, giving three. Choosing the top left greedily would block one of the bottom marks.

```
scan each row, starting a new run after every partition
scan each column in the same way
for every floor cell: connect its horizontal and vertical run IDs
answer = maximum matching size
```

6.3 Correctness proof

Any noninterfering placement uses each horizontal and vertical run at most once, so its cell edges form a matching of the same size. Conversely, a matching selects floor cells with no shared run. Two such cells cannot exchange an uninterrupted horizontal or vertical signal, so the placement is valid. The correspondence preserves the number of marks; a maximum matching gives an optimal placement.

6.4 Bounds, implementation notes, and tests

For `C` floor cells, there are at most `C` vertices on either side and exactly `C` edges. The included matcher uses `O(C^2+h*w)` time and `O(h*w)` memory. In C++, run IDs start at -1 for partitions. In Python, input rows are bytes: `35` is `#` and `46` is `.`. Start a new ID only at the first floor cell or after a partition, not after every cell.

The tiny oracle enumerates subsets of actual floor cells and checks direct visibility without run IDs or matching. Fixed tests include a greedy trap, all partitions, and one long run. Maximum-size tests use an open floor (answer 60) and a checkerboard of isolated cells (answer 1800).

6.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <cassert>
#include <queue>
#include <utility>
#include <vector>

struct BipartiteMatching {
    std::vector<std::vector<int>> adj;
    std::vector<int> left, right;

    BipartiteMatching(int left_count, int right_count)
        : adj(left_count), left(left_count, -1), right(right_count, -1) {}

    void add_edge(int u, int v) {
        assert(0 <= u && u < static_cast<int>(left.size()));
        assert(0 <= v && v < static_cast<int>(right.size()));
        adj[u].push_back(v);
    }

    bool augment(int root) {
        std::vector<int> previous(right.size(), -1);
        std::vector<bool> seen(left.size(), false);
        std::queue<int> queue;
        seen[root] = true;
        queue.push(root);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int v : adj[u]) {
                if (previous[v] != -1) continue;
                previous[v] = u;
                int owner = right[v];
                if (owner == -1) {
                    while (v != -1) {
                        u = previous[v];
                        int old = left[u];
                        left[u] = v;
                        right[v] = u;
                        v = old;
                    }
                    return true;
                }
                if (!seen[owner]) {
                    seen[owner] = true;
                    queue.push(owner);
                }
```

```
            }
        }
        return false;
    }

    std::vector<std::pair<int, int>> maximum_matching() {
        for (int root = 0; root < static_cast<int>(left.size()); ++root)
            if (left[root] == -1) augment(root);
        std::vector<std::pair<int, int>> pairs;
        for (int u = 0; u < static_cast<int>(left.size()); ++u)
            if (left[u] != -1) pairs.push_back({u, left[u]});
        return pairs;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int h, w;
    if (!(cin >> h >> w)) return 0;
    vector<string> grid(h);
    for (auto& row : grid) cin >> row;
    vector<vector<int>> row(h, vector<int>(w, -1));
    vector<vector<int>> col(h, vector<int>(w, -1));
    int a = 0, b = 0;
    for (int i = 0; i < h; ++i) {
        for (int j = 0; j < w; ++j) {
            if (grid[i][j] == '#') continue;
            if (j == 0 || grid[i][j - 1] == '#') ++a;
            row[i][j] = a - 1;
        }
    }
    for (int j = 0; j < w; ++j) {
        for (int i = 0; i < h; ++i) {
            if (grid[i][j] == '#') continue;
            if (i == 0 || grid[i - 1][j] == '#') ++b;
            col[i][j] = b - 1;
        }
    }
    BipartiteMatching matching(a, b);
    for (int i = 0; i < h; ++i)
        for (int j = 0; j < w; ++j)
            if (grid[i][j] == '.')
                matching.add_edge(row[i][j], col[i][j]);
    cout << matching.maximum_matching().size() << '\n';
}
```

6.6 Full Python solution

```python
import sys
from collections import deque


class BipartiteMatching:
    """Independent 0-based IDs on each side. -1 means unmatched."""

    def __init__(self, left_count, right_count):
        self.adj = [[] for _ in range(left_count)]
        self.left = [-1] * left_count
        self.right = [-1] * right_count

    def add_edge(self, u, v):
        assert 0 <= u < len(self.left) and 0 <= v < len(self.right)
        self.adj[u].append(v)

    def _augment(self, root):
        previous = [-1] * len(self.right)
        seen = [False] * len(self.left)
        seen[root] = True
        queue = deque([root])
        while queue:
            u = queue.popleft()
            for v in self.adj[u]:
                if previous[v] != -1:
                    continue
                previous[v] = u
                owner = self.right[v]
                if owner == -1:
                    # Follow predecessors and flip the entire path.
                    while v != -1:
                        u = previous[v]
                        old = self.left[u]
                        self.left[u] = v
                        self.right[v] = u
                        v = old
                    return True
                if not seen[owner]:
                    seen[owner] = True
                    queue.append(owner)
        return False

    def maximum_matching(self):
        for root in range(len(self.left)):
            if self.left[root] == -1:
                self._augment(root)
        return [(u, v) for u, v in enumerate(self.left) if v != -1]
```

```
def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    h, w = map(int, data[:2])
    grid = data[2:]
    row = [[-1] * w for _ in range(h)]
    col = [[-1] * w for _ in range(h)]
    a = b = 0
    for i in range(h):
        for j in range(w):
            if grid[i][j] == 35:
                continue
            if j == 0 or grid[i][j - 1] == 35:
                a += 1
            row[i][j] = a - 1
    for j in range(w):
        for i in range(h):
            if grid[i][j] == 35:
                continue
            if i == 0 or grid[i - 1][j] == 35:
                b += 1
            col[i][j] = b - 1
    matching = BipartiteMatching(a, b)
    for i in range(h):
        for j in range(w):
            if grid[i][j] == 46:
                matching.add_edge(row[i][j], col[i][j])
    print(len(matching.maximum_matching()))


if __name__ == '__main__':
    main()
```

7 F. Crate Routing

7.1 Restatement and model

A lane amount uses stock at one depot and capacity at one destination. Create one vertex per depot and destination. One unit of flow is one crate:

Edge	Capacity	Meaning
`s -> depot i`	`a[i]`	Do not ship more than its stock
`depot i -> destination j`	listed lane limit	Respect that direct lane
`destination j -> t`	`b[j]`	Do not exceed its receiving limit

These are capacities, not compulsory amounts. Unused stock is allowed.

7.2 Sample trace and algorithm

The sample's total stock and total receiving capacity are both 5, but depot 2 can ship only to destination 1, whose limit is 2. Depot 1 can use destination 2 for at most 2. Deliver 4 using those two lanes; a greedy shipment from depot 1 to destination 1 must be rerouted to achieve it.

```
build source/depot, lane, and destination/sink edges
print maximum_flow(s,t)
```

7.3 Correctness proof

For any valid shipment, put each lane amount on its middle edge, put each depot's total on its source edge, and each destination's total on its sink edge. All capacities hold and conservation follows from the totals, giving a feasible flow of the same delivered value. Conversely, integral flow conservation makes middle-edge amounts a valid shipment. Source and sink capacities enforce both endpoint limits, and the middle capacities enforce lane limits. Dinic preserves integrality, so its maximum flow corresponds to a maximum integer shipment.

7.4 Bounds, implementation notes, and tests

`V=n+m+2`, `E=n+m+e`: time `O(V^2 E)`, space `O(V+E)`. The answer can be `150*10^9`. Use `long long` for capacities and totals in C++; Python reads all integers before building the graph. There is no depot-to-depot routing. Do not omit lane limits or mistake a destination limit for an exact demand.

The independent oracle enumerates lane amounts on tiny cases, checking original stocks and receiving limits. Tests cover zero stock, no lanes, a lane bottleneck, and rerouting. Dense 150-by-150 stress cases check large totals and the case where only one destination can receive anything.

7.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
```

```
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
```

```
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e;
    if (!(cin >> n >> m >> e)) return 0;
    int s = n + m, t = s + 1;
    Dinic flow(t + 1);
    for (int i = 0; i < n; ++i) {
        long long x; cin >> x; flow.add_edge(s, i, x);
    }
    for (int j = 0; j < m; ++j) {
        long long x; cin >> x; flow.add_edge(n + j, t, x);
    }
    while (e--) {
        int u, v; long long c; cin >> u >> v >> c;
        flow.add_edge(u - 1, n + v - 1, c);
    }
    cout << flow.max_flow(s, t) << '\n';
```

```
}
```

7.6 Full Python solution

```python
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
```

```
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
```

```
        return seen


def main():
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, m, e = next(it), next(it), next(it)
    s, t = n + m, n + m + 1
    flow = Dinic(t + 1)
    for i in range(n):
        flow.add_edge(s, i, next(it))
    for j in range(m):
        flow.add_edge(n + j, t, next(it))
    for _ in range(e):
        u, v, c = next(it), next(it), next(it)
        flow.add_edge(u - 1, n + v - 1, c)
    print(flow.max_flow(s, t))


if __name__ == '__main__':
    main()
```

8 G. Silent Rehearsal

8.1 Restatement and threshold model

We minimize the largest chosen preparation time, not their sum. Fix a waiting time `T`. Keep exactly the performer-room pairs with time at most `T`. A matching of size `n` is precisely a complete assignment ready by `T`.

Increasing `T` only adds edges, so feasibility is monotone. It changes only at a listed time. Sort and deduplicate the times, check the largest, then binary-search the first feasible listed value. If there are no edges, or even all edges cannot match every performer, print -1.

8.2 Sample trace and algorithm

At time 2 in the sample, both performers can use only room 1. At time 4, performer 1 also gains room 2; the assignment (1,2), (2,1) becomes possible. The smallest waiting time is 4, even though the individually fastest pair for performer 1 would use room 1.

```
if no full matching using all listed pairs: return -1
binary-search distinct listed times T
    build pairs whose preparation time <= T
    test whether matching size is n
return the first feasible T
```

8.3 Correctness proof

For a fixed `T`, an assignment with waiting time at most `T` selects `n` disjoint edges in the threshold graph. A size-`n` matching conversely gives such an assignment. If an assignment is feasible at one threshold, it remains feasible at every larger one. Any optimal assignment's maximum chosen time is one of the listed values. Binary search over those values therefore returns exactly the smallest possible waiting time.

8.4 Bounds, implementation notes, and tests

There are `O(log(e+1))` matching calls. With the provided BFS matcher, time is `O(n*(e+n+m)*log(e+1) + e*log(e+1))`; memory is `O(e+n+m)`. Rebuild the matcher for each candidate so old assignments using forbidden edges cannot leak into a lower threshold. Zero times are valid. Having at least `n` rooms or one choice per performer is not enough: groups may compete for the same small set.

The oracle enumerates injective assignments and minimizes their actual maximum time, without binary search. Stress tests include 62,500 distinct edge times: a linear scan rebuilding a matcher

for every time is unsuitable. For times `u*(n+1)+v` with 0-based indices, the last performer needs at least `(n-1)*(n+1)=n^2-1`; give it room 0 and give earlier performers the other rooms to attain that bound. A second dense test has a zero-time diagonal.

8.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

#include <cassert>
#include <queue>
#include <utility>
#include <vector>

struct BipartiteMatching {
    std::vector<std::vector<int>> adj;
    std::vector<int> left, right;

    BipartiteMatching(int left_count, int right_count)
        : adj(left_count), left(left_count, -1), right(right_count, -1) {}

    void add_edge(int u, int v) {
        assert(0 <= u && u < static_cast<int>(left.size()));
        assert(0 <= v && v < static_cast<int>(right.size()));
        adj[u].push_back(v);
    }

    bool augment(int root) {
        std::vector<int> previous(right.size(), -1);
        std::vector<bool> seen(left.size(), false);
        std::queue<int> queue;
        seen[root] = true;
        queue.push(root);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int v : adj[u]) {
                if (previous[v] != -1) continue;
                previous[v] = u;
                int owner = right[v];
                if (owner == -1) {
                    while (v != -1) {
                        u = previous[v];
                        int old = left[u];
                        left[u] = v;
                        right[v] = u;
                        v = old;
                    }
                    return true;
```

```
                }
                if (!seen[owner]) {
                    seen[owner] = true;
                    queue.push(owner);
                }
            }
        }
        return false;
    }

    std::vector<std::pair<int, int>> maximum_matching() {
        for (int root = 0; root < static_cast<int>(left.size()); ++root)
            if (left[root] == -1) augment(root);
        std::vector<std::pair<int, int>> pairs;
        for (int u = 0; u < static_cast<int>(left.size()); ++u)
            if (left[u] != -1) pairs.push_back({u, left[u]});
        return pairs;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e;
    if (!(cin >> n >> m >> e)) return 0;
    vector<tuple<int, int, long long>> edges(e);
    vector<long long> times;
    for (auto& [u, v, d] : edges) {
        cin >> u >> v >> d; --u; --v; times.push_back(d);
    }
    sort(times.begin(), times.end());
    times.erase(unique(times.begin(), times.end()), times.end());
    auto feasible = [&](long long limit) {
        BipartiteMatching matching(n, m);
        for (auto [u, v, d] : edges)
            if (d <= limit) matching.add_edge(u, v);
        return static_cast<int>(matching.maximum_matching().size()) == n;
    };
    if (times.empty() || !feasible(times.back())) {
        cout << -1 << '\n'; return 0;
    }
    int lo = 0, hi = static_cast<int>(times.size()) - 1;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (feasible(times[mid])) hi = mid;
        else lo = mid + 1;
    }
    cout << times[lo] << '\n';
}
```

8.6 Full Python solution

```python
import sys
from collections import deque


class BipartiteMatching:
    """Independent 0-based IDs on each side. -1 means unmatched."""

    def __init__(self, left_count, right_count):
        self.adj = [[] for _ in range(left_count)]
        self.left = [-1] * left_count
        self.right = [-1] * right_count

    def add_edge(self, u, v):
        assert 0 <= u < len(self.left) and 0 <= v < len(self.right)
        self.adj[u].append(v)

    def _augment(self, root):
        previous = [-1] * len(self.right)
        seen = [False] * len(self.left)
        seen[root] = True
        queue = deque([root])
        while queue:
            u = queue.popleft()
            for v in self.adj[u]:
                if previous[v] != -1:
                    continue
                previous[v] = u
                owner = self.right[v]
                if owner == -1:
                    # Follow predecessors and flip the entire path.
                    while v != -1:
                        u = previous[v]
                        old = self.left[u]
                        self.left[u] = v
                        self.right[v] = u
                        v = old
                    return True
                if not seen[owner]:
                    seen[owner] = True
                    queue.append(owner)
        return False

    def maximum_matching(self):
        for root in range(len(self.left)):
            if self.left[root] == -1:
                self._augment(root)
        return [(u, v) for u, v in enumerate(self.left) if v != -1]
```

```
def main():
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, m, e = next(it), next(it), next(it)
    edges = [(next(it) - 1, next(it) - 1, next(it)) for _ in range(e)]
    times = sorted({d for u, v, d in edges})
    def feasible(limit):
        matching = BipartiteMatching(n, m)
        for u, v, d in edges:
            if d <= limit:
                matching.add_edge(u, v)
        return len(matching.maximum_matching()) == n
    if not times or not feasible(times[-1]):
        print(-1)
        return
    lo, hi = 0, len(times) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(times[mid]):
            hi = mid
        else:
            lo = mid + 1
    print(times[lo])


if __name__ == '__main__':
    main()
```

9 H. Emergency Corridors

9.1 Restatement and modeling obstacle

We pay for rooms, not for corridors. Giving each corridor the cost of its endpoint would charge multiple times for a room shared by several paths. Instead split each room `v` into `v_in` and `v_out`:

```
v_in -> v_out: closure cost of v
u_out -> v_in: INF for each input corridor u -> v
s -> hazard_in: INF
archive_out -> t: INF
```

Protected rooms receive `INF` on their internal edge. Choose `INF = 1 + sum of all nonnegative listed costs`. Closing every allowed nonterminal room costs less than `INF`, so any feasible finite optimum cannot afford a protected edge or corridor.

9.2 Trace and algorithm

In the sample, one hazard-to-archive route passes through room 2, and the other through rooms 3 and 4. Delete room 2 for 4, then choose the cheaper of room 3 for 2 and room 4 for 7. The answer is 6. The internal edge for a room is shared by every route through it, so a closure is paid only once.

```
split rooms and build the stated network
value = maximum_flow(s,t)
if value >= INF: print -1
else: print value
```

9.3 Correctness proof

A valid closure set disconnects source and sink after the corresponding internal edges are removed. Let the source side be the vertices reachable without those edges. Its cut crosses only removed internal edges and costs at most the closure set. Thus minimum cut is no more expensive than any valid closure.

Conversely, a cut costing less than `INF` crosses only internal edges of removable rooms. Close exactly those rooms. Any remaining hazard-to-archive path would induce a source-to-sink path in the split graph and would have to cross the cut without using a closed internal edge, which is impossible. These closures are valid and have the cut's cost. Therefore finite minimum cut equals minimum closure cost. If the minimum cut is at least `INF`, a valid closure set cannot exist, since all allowed closures together cost less than that bound.

9.4 Bounds, implementation notes, and tests

`V=2n+2`, `E=n+m+k+q`: `O(V^2 E)` time and `O(V+E)` space. Input direction matters. Terminals are protected even if their listed costs are nonnegative. Zero-cost internal edges are meaningful and may produce answer zero. C++ stores the sum and `INF` in 64 bits; Python's explicit stack avoids recursion through the split graph.

The oracle enumerates removable-room subsets and performs reachability in the original building. It does not enumerate cuts in the transformed graph. Stress tests force all routes through a common room, or add one completely protected route that makes the task impossible.

9.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
```

```
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }
```

```
    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k, q;
    if (!(cin >> n >> m >> k >> q)) return 0;
    vector<long long> cost(n);
    long long budget = 0;
    for (auto& x : cost) { cin >> x; if (x >= 0) budget += x; }
    vector<int> starts(k), targets(q);
    vector<bool> protected_room(n);
    for (int& v : starts) { cin >> v; --v; protected_room[v] = true; }
    for (int& v : targets) { cin >> v; --v; protected_room[v] = true; }
    long long inf = budget + 1;
    int s = 2 * n, t = s + 1;
    Dinic flow(t + 1);
    for (int v = 0; v < n; ++v) {
        long long c = cost[v] < 0 || protected_room[v] ? inf : cost[v];
        flow.add_edge(2 * v, 2 * v + 1, c);
    }
```

```
    for (int v : starts) flow.add_edge(s, 2 * v, inf);
    for (int v : targets) flow.add_edge(2 * v + 1, t, inf);
    while (m--) {
        int u, v; cin >> u >> v;
        flow.add_edge(2 * (u - 1) + 1, 2 * (v - 1), inf);
    }
    long long answer = flow.max_flow(s, t);
    cout << (answer >= inf ? -1 : answer) << '\n';
}
```

9.6 Full Python solution

```
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
```

```
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
```

```
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen


def main():
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, m, k, q = next(it), next(it), next(it), next(it)
    costs = [next(it) for _ in range(n)]
    starts = [next(it) - 1 for _ in range(k)]
    targets = [next(it) - 1 for _ in range(q)]
    protected = set(starts + targets)
    inf = sum(c for c in costs if c >= 0) + 1
    s, t = 2 * n, 2 * n + 1
    flow = Dinic(t + 1)
    for v, cost in enumerate(costs):
        capacity = inf if cost < 0 or v in protected else cost
        flow.add_edge(2 * v, 2 * v + 1, capacity)
    for v in starts:
        flow.add_edge(s, 2 * v, inf)
    for v in targets:
        flow.add_edge(2 * v + 1, t, inf)
    for _ in range(m):
        u, v = next(it) - 1, next(it) - 1
        flow.add_edge(2 * u + 1, 2 * v, inf)
    answer = flow.max_flow(s, t)
    print(-1 if answer >= inf else answer)


if __name__ == '__main__':
    main()
```

10 I. Directional Relay Modes

10.1 Restatement and cut convention

We pay one unary cost per relay and a directional penalty for each listed 0-to-1 pair. Interpret the source side as mode 0, and the sink side as mode 1. Then add:

```
s -> i with capacity c1[i]
i -> t with capacity c0[i]
u -> v with capacity p for notice (u,v,p)
```

The reversed-looking unary capacities are intentional: putting `i` with the sink cuts its source edge and pays mode 1's cost; putting it with the source cuts its sink edge and pays mode 0's cost.

10.2 Truth table and sample

Mode of u	Mode of v	Does `u -> v` cross the cut?
0	0	No
0	1	Yes: pay p
1	0	No
1	1	No

The sample's independently cheapest modes (0,1) incur penalty 10. Switching the second relay to 0 gives cost 4 and no crossing notice. An undirected penalty would also charge the opposite mismatch, incorrectly changing the problem. Run one maximum flow and print its value.

10.3 Correctness proof

Every assignment determines a cut. Exactly one unary edge per relay is cut, paying its chosen mode's running cost. Each notice edge crosses exactly in its triggering 0-to-1 case, so cut capacity equals total cost. Every cut conversely determines one mode per relay and obeys the same identity. Therefore minimizing cut capacity is precisely minimizing the stated objective, and max-flow/min-cut gives the answer.

10.4 Bounds, implementation notes, and tests

`V=n+2`, `E=2n+m`: `O(V^2 E)` time and `O(V+E)` space. Use 64-bit totals. No infinite edges are needed, because notices are penalties rather than forbidden assignments. Opposite notices have separate meanings. Empty source-side or sink-side relay sets are allowed.

The oracle enumerates all binary mode vectors and charges notices directly. A fixed case makes (1,0) optimal, catching accidental undirected penalties. Stress tests include a forced uniform choice where crossing any boundary costs a billion, but choosing all-zero or all-one costs only 125.

10.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
```

```
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }
```

```
    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    int s = n, t = n + 1;
    Dinic flow(n + 2);
    for (int i = 0; i < n; ++i) {
        long long c0, c1; cin >> c0 >> c1;
        flow.add_edge(s, i, c1);
        flow.add_edge(i, t, c0);
    }
    while (m--) {
        int u, v; long long penalty; cin >> u >> v >> penalty;
        flow.add_edge(u - 1, v - 1, penalty);
    }
    cout << flow.max_flow(s, t) << '\n';
}
```

10.6 Full Python solution

```
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
```

```
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen


def main():
```

```
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, m = next(it), next(it)
    s, t = n, n + 1
    flow = Dinic(n + 2)
    for i in range(n):
        c0, c1 = next(it), next(it)
        flow.add_edge(s, i, c1)
        flow.add_edge(i, t, c0)
    for _ in range(m):
        u, v, penalty = next(it), next(it), next(it)
        flow.add_edge(u - 1, v - 1, penalty)
    print(flow.max_flow(s, t))


if __name__ == '__main__':
    main()
```

11 J. Exhibit Pacts

11.1 Restatement and why one auxiliary vertex helps

The reward for a group is paid once when every member has the requested state. Pairwise edges alone would generally count a bonus several times. Use source-side exhibits for open, sink-side exhibits for closed. Let `P = sum of positive exhibit values + sum of every pact bonus`. The cut will measure reward lost from this optimistic baseline.

Use ordinary profit edges for exhibit values: `s -> i` for positive value, `i -> t` for its absolute negative value. For each pact add one auxiliary vertex `a`, using `INF=P+1`:

Pact	Bonus edge	Requirement edges
All open, bonus b	`s -> a`, capacity b	`a -> member`, capacity INF
All closed, bonus b	`a -> t`, capacity b	`member -> a`, capacity INF

11.2 What each auxiliary choice means

For an all-open pact, placing `a` with the source claims its bonus: all members must also be with the source, or an infinite edge crosses. If any member is closed, `a` must be with the sink and the cut loses `b` once. For an all-closed pact, placing `a` with the sink claims the bonus. Any open member forces `a` to the source, losing `b` once. If all members are closed, placing `a` with the sink avoids that loss. All pacts get independent auxiliary vertices, so overlapping or repeated groups are handled naturally.

11.3 Sample trace and algorithm

For opening only exhibit 1, baseline `P=4+2+6+5=17`. The cut loses exhibit 3's positive value 2 and the unsatisfied all-open bonus 6, totaling 8. It retains the all-closed bonus 5, and `17-8=9`. Opening all exhibits would instead earn only 7.

```
compute P and INF=P+1
build exhibit profit edges and one gadget per pact
print P - maximum_flow(s,t)
```

11.4 Correctness proof

A consistent exhibit selection can place each auxiliary vertex to claim exactly its satisfied pact bonuses. The cut then pays every unselected positive value, every selected negative cost, and every unsatisfied bonus: its capacity is `P - total_value`. Conversely, a cut of capacity at most `P` exists by taking any selection and the consistent auxiliaries. Hence no minimum cut crosses `INF`. Its finite auxiliary choices obey all stated requirements, and optimizing them for fixed exhibits loses exactly the unsatisfied bonuses. Minimizing the cut therefore maximizes total value.

11.5 Bounds, implementation notes, and tests

With total membership count `z`, `V=n+g+2` and `E=O(n+g+z)`. Dinic takes `O(V^2 E)` time and `O(V+E)` memory. Store signed values and `P` in 64 bits in C++; Python needs no integer-size changes. Do not add infinite edges directly between all exhibit members: disagreement merely loses the bonus, it is not forbidden. A zero bonus still gives a valid harmless gadget.

The oracle enumerates exhibit subsets and tests each original pact directly. Fixed tests include conflicting pacts on the same group and single-member pacts. Maximum-size all-open and all-closed cases test all 3000 memberships and answers exceeding 32-bit range.

11.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
```

```
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
```

```
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, g;
    if (!(cin >> n >> g)) return 0;
    vector<long long> value(n), bonus(g);
    vector<int> type(g);
    vector<vector<int>> members(g);
```

```
    long long base = 0;
    for (auto& x : value) { cin >> x; base += max(0LL, x); }
    for (int k = 0; k < g; ++k) {
        int size; cin >> type[k] >> bonus[k] >> size;
        base += bonus[k]; members[k].resize(size);
        for (int& v : members[k]) { cin >> v; --v; }
    }
    int s = n + g, t = s + 1;
    Dinic flow(t + 1);
    long long inf = base + 1;
    for (int i = 0; i < n; ++i) {
        if (value[i] >= 0) flow.add_edge(s, i, value[i]);
        else flow.add_edge(i, t, -value[i]);
    }
    for (int k = 0; k < g; ++k) {
        int a = n + k;
        if (type[k] == 1) {
            flow.add_edge(s, a, bonus[k]);
            for (int v : members[k]) flow.add_edge(a, v, inf);
        } else {
            flow.add_edge(a, t, bonus[k]);
            for (int v : members[k]) flow.add_edge(v, a, inf);
        }
    }
    cout << base - flow.max_flow(s, t) << '\n';
}
```

11.7 Full Python solution

```
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
```

```
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0
```

```
    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen


def main():
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, g = next(it), next(it)
    profits = [next(it) for _ in range(n)]
    groups = []
    base = sum(max(0, v) for v in profits)
    for _ in range(g):
        kind, bonus, size = next(it), next(it), next(it)
        group = [next(it) - 1 for _ in range(size)]
        groups.append((kind, bonus, group))
        base += bonus
    s, t = n + g, n + g + 1
    flow = Dinic(t + 1)
    inf = base + 1
    for i, value in enumerate(profits):
        if value >= 0:
            flow.add_edge(s, i, value)
        else:
            flow.add_edge(i, t, -value)
    for k, (kind, bonus, group) in enumerate(groups):
```

```
        a = n + k
        if kind == 1:
            flow.add_edge(s, a, bonus)
            for v in group:
                flow.add_edge(a, v, inf)
        else:
            flow.add_edge(a, t, bonus)
            for v in group:
                flow.add_edge(v, a, inf)
    print(base - flow.max_flow(s, t))


if __name__ == '__main__':
    main()
```

12 K. Quota Manifests

12.1 Restatement and bounded network

This is feasibility with compulsory minimums, not a maximum-delivery question. Use source `s`, depot vertices, destination vertices, and sink `t`:

```
s -> depot i: [lo[i], hi[i]]
depot i -> destination j: [0, lane_capacity]
destination j -> t: [low[j], high[j]]
t -> s: [K,K]
```

The last edge returns exactly the required delivery and turns the whole network into a circulation. It is an additional input edge, not a residual reverse entry. Every vertex, including `s` and `t`, must now conserve flow.

12.2 Convert lower bounds to ordinary flow

For bounded edge `u -> v` with `[l,h]`, preassign `l`, add optional capacity `h-l`, subtract `l` from `balance[u]`, and add it to `balance[v]`. Positive balance means excess compulsory inflow that optional flow must send onward. Negative balance means missing inflow. Add auxiliary terminals:

```
balance[v] > 0: SS -> v of capacity balance[v]
balance[v] < 0: v -> TT of capacity -balance[v]
required = sum of positive balances
answer YES iff maximum_flow(SS,TT) == required
```

The lesson's bounded-circulation primer proves these signs. In particular, `SS -> v` forces extra original outflow, rather than describing an original external supply. Reversing the auxiliary directions is a common error.

12.3 Sample trace and correctness

The sample can put one crate on each of its three lanes. Depot totals are 2 and 1, destination totals are 2 and 1, and the return edge carries 3. All bounds and conservation hold, so every auxiliary demand can be met.

Generally, any manifest extends to a bounded circulation by its endpoint totals and the fixed return amount. Conversely, any such circulation has exactly `K` on depot-to-destination lanes and satisfies all original quotas. Subtracting compulsory lower amounts from a valid circulation gives optional amounts within capacities that, together with saturated auxiliary edges, conserve at

every vertex. Conversely, saturation of every auxiliary source edge repairs all balances; adding back lower amounts gives a valid circulation. The balances sum to zero, so meeting total positive demand also meets all negative demand. These equivalences prove the decision rule.

12.4 Bounds, implementation notes, and tests

`V=n+m+4`, `E=O(n+m+e)` after auxiliaries. Time is `O(V^2 E)` and memory `O(V+E)`. The total may be `10^11`, so balances and `required` use 64-bit integers in C++. A zero-capacity optional edge is valid when `lo=hi`. No artificial infinity is required. Checking only sums of lower and upper bounds misses missing-lane obstructions; treating lower bounds as capacities misses compulsory deliveries.

The independent oracle enumerates lane amounts, then checks all original row intervals, column intervals, and the total. Stress cases use fixed billion-unit quotas or a disconnected destination even though aggregate upper bounds seem sufficient.

12.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }
```

```
bool bfs(int s, int t) {
    level.assign(n, -1);
    level[s] = 0;
    std::queue<int> queue;
    queue.push(s);
    while (!queue.empty()) {
        int u = queue.front();
        queue.pop();
        for (int edge : adj[u]) {
            int v = edges[edge].to;
            if (edges[edge].cap > 0 && level[v] == -1) {
                level[v] = level[u] + 1;
                queue.push(v);
            }
        }
    }
    return level[t] != -1;
}

long long send(int s, int t, long long limit) {
    std::vector<int> vertices{s}, path;
    std::vector<long long> bottleneck{limit};
    while (!vertices.empty()) {
        int u = vertices.back();
        if (u == t) {
            long long delta = bottleneck.back();
            for (int edge : path) {
                edges[edge].cap -= delta;
                edges[edge ^ 1].cap += delta;
            }
            return delta;
        }
        while (ptr[u] < static_cast<int>(adj[u].size())) {
            int edge = adj[u][ptr[u]];
            int v = edges[edge].to;
            if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                break;
            ++ptr[u];
        }
        if (ptr[u] == static_cast<int>(adj[u].size())) {
            vertices.pop_back();
            bottleneck.pop_back();
            if (!path.empty()) {
                path.pop_back();
                ++ptr[vertices.back()];
            }
        } else {
            int edge = adj[u][ptr[u]];
            path.push_back(edge);
            vertices.push_back(edges[edge].to);
```

```
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, e;
    long long total;
    if (!(cin >> n >> m >> e >> total)) return 0;
    int s = n + m, t = s + 1, ss = t + 1, tt = ss + 1;
    Dinic flow(tt + 1);
    vector<long long> balance(t + 1);
    auto bounded = [&](int u, int v, long long lo, long long hi) {
        flow.add_edge(u, v, hi - lo);
        balance[u] -= lo;
        balance[v] += lo;
```

```
    };
    for (int i = 0; i < n; ++i) {
        long long lo, hi; cin >> lo >> hi;
        bounded(s, i, lo, hi);
    }
    for (int j = 0; j < m; ++j) {
        long long lo, hi; cin >> lo >> hi;
        bounded(n + j, t, lo, hi);
    }
    while (e--) {
        int u, v; long long c; cin >> u >> v >> c;
        bounded(u - 1, n + v - 1, 0, c);
    }
    bounded(t, s, total, total);
    long long required = 0;
    for (int v = 0; v <= t; ++v) {
        if (balance[v] > 0) {
            flow.add_edge(ss, v, balance[v]);
            required += balance[v];
        } else if (balance[v] < 0) {
            flow.add_edge(v, tt, -balance[v]);
        }
    }
    cout << (flow.max_flow(ss, tt) == required ? "YES" : "NO") << '\n';
}
```

12.6 Full Python solution

```
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
```

```
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
```

```
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen


def main():
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, m, e, total = next(it), next(it), next(it), next(it)
    s, t = n + m, n + m + 1
    ss, tt = t + 1, t + 2
    flow = Dinic(tt + 1)
    balance = [0] * (t + 1)
    def bounded(u, v, lo, hi):
        flow.add_edge(u, v, hi - lo)
        balance[u] -= lo
        balance[v] += lo
    for i in range(n):
        lo, hi = next(it), next(it)
        bounded(s, i, lo, hi)
    for j in range(m):
        lo, hi = next(it), next(it)
        bounded(n + j, t, lo, hi)
    for _ in range(e):
        u, v, c = next(it), next(it), next(it)
        bounded(u - 1, n + v - 1, 0, c)
    bounded(t, s, total, total)
```

```
    required = 0
    for v, value in enumerate(balance):
        if value > 0:
            flow.add_edge(ss, v, value)
            required += value
        elif value < 0:
            flow.add_edge(v, tt, -value)
    print('YES' if flow.max_flow(ss, tt) == required else 'NO')


if __name__ == '__main__':
    main()
```

13 L. Calendar Reinforcement

13.1 Restatement and monotonicity

Choose a uniform extra daily staffing level. A fixed level `x` is feasible if all jobs can be scheduled, with at most one unit of each job per day. More staff cannot hurt. If `work > deadline-release` for any job, no amount of staff can help; print -1. Otherwise `n` extra workers suffice: reserve one extra worker for each job and choose any required days in its window. Binary-search `x` in `[0,n]`.

13.2 Compress equivalent days

Enumerate all calendar boundaries and all job releases and deadlines. Between two consecutive boundaries, staff capacity is constant and the same jobs are available. Replace those integer days by one block of length `len`. For fixed `x`, create:

```
s -> job i: work[i]
job i -> contained block j: len[j]
block j -> t: (base[j]+x)*len[j]
```

The middle capacity is **length**, not infinity. Even with many workers, one job can perform at most one unit per day. The sink capacity limits all jobs together. Feasibility means maximum flow equals total required work; source saturation then completes every job exactly.

13.3 Why aggregation does not invent an impossible schedule

A real schedule gives integral amounts on job-to-block edges and respects both kinds of capacity, hence gives the required flow. For the reverse direction, consider one block of `len` days. Its flow assigns integer amounts `y[i]` with `0 <= y[i] <= len` and `sum y[i] <= staff*len`.

These amounts always fit actual days. Number the days `0,...,len-1`, and write job 1's `y[1]` units on consecutive cyclic day positions, then continue from the next position for job 2, and so on. No job repeats a day because its amount is at most `len`. Across all jobs, each day receives at most `ceil(sum y[i]/len) <= staff` units. Every job is available throughout the block, so these placements are legal. Apply this separately to every block. Thus the compressed feasibility test is exact, including the per-job daily restriction.

13.4 Sample trace, algorithm, and correctness

With `x=0`, the sample can process only two total units, below the required five. With `x=1`, daily capacities are 2,2,1,1. Jobs can use days (0,1), (2,3), and (0), respectively. The test becomes feasible, giving answer 1.

```
reject any job with work > window length
split the calendar at all calendar and job endpoints
binary-search x in [0,n]
    build a fresh job/block network
    feasible iff max_flow equals total work
```

The schedule/flow correspondence proves every test. Feasibility is monotone and the upper bound is feasible, so binary search returns the smallest additional staff level.

13.5 Bounds, implementation notes, and tests

There are at most `B=2n+p` blocks. Each test has `V=n+B+2` and `E=O(n*B+n+B)`; there are `O(log(n+1))` tests. General time is `O(log(n+1)*V^2 E)`, space `O(n*B+n+B)`. Use half-open windows and 64-bit lengths/products in C++. The largest raw block capacity is below `1.000001*10^18`; the reference also clamps it to total required work, since extra capacity beyond that cannot affect feasibility. Python uses the same clamping and iterative Dinic.

Do not expand one vertex per day. Do not permit multiple workers to process the same job simultaneously. The oracle enumerates actual day subsets per job on calendars of at most five days, without compression or flow. Large stress cases force 80 simultaneous full-window jobs, and introduce nearly all possible distinct compression boundaries with billion-day windows.

13.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;
```

```
    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
```

```
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, p;
    if (!(cin >> n >> p)) return 0;
    vector<array<long long, 3>> calendar(p), jobs(n);
    vector<long long> points;
    for (auto& [l, r, c] : calendar) {
        cin >> l >> r >> c; points.push_back(l); points.push_back(r);
    }
    long long total = 0;
    bool impossible = false;
    for (auto& [l, r, work] : jobs) {
        cin >> l >> r >> work;
        points.push_back(l); points.push_back(r); total += work;
        if (work > r - l) impossible = true;
    }
    if (impossible) { cout << -1 << '\n'; return 0; }
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()), points.end());
    int blocks = static_cast<int>(points.size()) - 1;
    vector<long long> base(blocks);
    int current = 0;
    for (int j = 0; j < blocks; ++j) {
        while (points[j] >= calendar[current][1]) ++current;
        base[j] = calendar[current][2];
    }
    auto feasible = [&](int extra) {
        int s = n + blocks, t = s + 1;
        Dinic flow(t + 1);
        for (int i = 0; i < n; ++i) {
            auto [l, r, work] = jobs[i];
            flow.add_edge(s, i, work);
            for (int j = 0; j < blocks; ++j)
                if (l <= points[j] && points[j + 1] <= r)
                    flow.add_edge(i, n + j, points[j + 1] - points[j]);
        }
        for (int j = 0; j < blocks; ++j) {
            long long length = points[j + 1] - points[j];
            long long capacity = min(total, (base[j] + extra) * length);
            flow.add_edge(n + j, t, capacity);
        }
        return flow.max_flow(s, t) == total;
    };
    int lo = 0, hi = n;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (feasible(mid)) hi = mid;
        else lo = mid + 1;
    }
    cout << lo << '\n';
```

```
}
```

13.7 Full Python solution

```python
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
```

```
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
```

```
        return seen


def main():
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, p = next(it), next(it)
    calendar = [(next(it), next(it), next(it)) for _ in range(p)]
    jobs = [(next(it), next(it), next(it)) for _ in range(n)]
    if any(work > r - l for l, r, work in jobs):
        print(-1)
        return
    points = sorted({x for l, r, c in calendar + jobs for x in (l, r)})
    blocks = len(points) - 1
    base = []
    current = 0
    for x in points[:-1]:
        while x >= calendar[current][1]:
            current += 1
        base.append(calendar[current][2])
    total = sum(work for l, r, work in jobs)
    def feasible(extra):
        s, t = n + blocks, n + blocks + 1
        flow = Dinic(t + 1)
        for i, (l, r, work) in enumerate(jobs):
            flow.add_edge(s, i, work)
            for j in range(blocks):
                if l <= points[j] and points[j + 1] <= r:
                    flow.add_edge(i, n + j, points[j + 1] - points[j])
        for j in range(blocks):
            length = points[j + 1] - points[j]
            flow.add_edge(n + j, t, min(total, (base[j] + extra) * length))
        return flow.max_flow(s, t) == total
    lo, hi = 0, n
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid):
            hi = mid
        else:
            lo = mid + 1
    print(lo)


if __name__ == '__main__':
    main()
```

14 M. Resilient Reservations

14.1 Restatement and the right adversarial condition

An assignment can be recomputed after cancellation. Counting unused slots in one matching therefore does not measure resilience. Hall's theorem says all visitors can be assigned exactly when every visitor group `X` has at least `|X|` neighboring slots.

For a nonempty `X`, an adversary can remove up to `k` slots from `N(X)`. Every deletion of at most `k` is survivable exactly when

```
|N(X)| >= |X| + k for every nonempty visitor set X.
```

Sufficiency: any such deletion leaves at least `|X|` neighbors for every group, so Hall still holds. Necessity: if `|N(X)| < |X|+k`, either the original assignment already fails, or deleting `|N(X)|-|X|+1 <= k` of its neighbors makes that group deficient. Hence the answer is the minimum slack `|N(X)|-|X|` over nonempty sets, or -1 if this minimum is negative.

14.2 Find minimum slack without enumerating groups

Build a cut network with source-to-visitor capacity 1, compatibility visitor-to-slot capacity `INF=n+m+1`, and slot-to-sink capacity 1. For a source-side visitor set `X`, a finite cut must put every slot of `N(X)` with the source too. Including any extra slot only increases cost, so the cheapest cut for that `X` has capacity

```
(n - |X|) + |N(X)| = n + slack(X).
```

The empty set has slack zero and would hide positive resilience. To exclude it, try each visitor `r` as a mandatory member of `X`: add `s -> r` with capacity `INF`. A minimum cut cannot cross it, since putting all visitors and slots with the source costs only `m < INF`. Subtract `n` from that minimum cut value, and minimize over the `n` choices of `r`. Every nonempty set contains at least one possible forced visitor, so all are covered.

14.3 Sample trace and algorithm

In the sample, group `{visitor 1}` has two neighbors, giving slack 1. Visitor 2 alone has slack 2, and both together have slack 2. The minimum is 1. In contrast, two visitors both accepting only slots 1 and 2 have slack zero together even though each individual has degree two. Checking minimum degree alone misses this collective obstruction.

```
answer = m
for each visitor r:
    build the forced-r cut network
    answer = min(answer, max_flow(s,t) - n)
print -1 if answer < 0 else answer
```

The Hall argument characterizes resilience; the forced-cut identity computes exactly its minimum slack. Together they prove the result. Cancellation of slots accepted by nobody cannot worsen the guarantee.

14.4 Bounds, implementation notes, and tests

There are `n` flow computations on `V=n+m+2`, `E=O(n+m+e)` networks. The general bound is `O(n*V^2 E)` and space is `O(n+m+e)`. Here each flow is at most `m`, since there are only `m` unit sink edges. Integral pushes and at most `m` successful phases also give the more informative bound `O(n*m*(V+E))` for these particular networks. The supplied limits keep rebuilding practical in both languages.

Compatibility capacities must be `INF`, not 1: they enforce membership of every neighbor on the cut side, rather than represent one matched pair. Keep the ordinary capacity-1 source edges even for the forced visitor. Use an additional large source edge to impose its side. A negative slack means initial failure, not zero resilience.

The independent oracle enumerates cancellation sets and injective assignments on the remaining slots. It uses neither Hall slack nor cuts. Dense stress cases have known resilience `m-n`, or one visitor with only one slot, forcing resilience zero despite many spare slots elsewhere.

14.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}
```

```
    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
```

```
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    int n, m, e;
    if (!(cin >> n >> m >> e)) return 0;
    vector<pair<int, int>> edges(e);
    for (auto& [u, v] : edges) { cin >> u >> v; --u; --v; }
    int answer = m;
    long long inf = n + m + 1;
    for (int forced = 0; forced < n; ++forced) {
        int s = n + m, t = s + 1;
        Dinic flow(t + 1);
        for (int u = 0; u < n; ++u) flow.add_edge(s, u, 1);
        for (int v = 0; v < m; ++v) flow.add_edge(n + v, t, 1);
        for (auto [u, v] : edges) flow.add_edge(u, n + v, inf);
        flow.add_edge(s, forced, inf);
        int slack = static_cast<int>(flow.max_flow(s, t)) - n;
        answer = min(answer, slack);
        if (answer < 0) break;
    }
    cout << (answer < 0 ? -1 : answer) << '\n';
}
```

14.6 Full Python solution

```
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
```

```
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
```

```
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen


def main():
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, m, e = next(it), next(it), next(it)
    edges = [(next(it) - 1, next(it) - 1) for _ in range(e)]
    inf = n + m + 1
    answer = m
    for forced in range(n):
        s, t = n + m, n + m + 1
        flow = Dinic(t + 1)
        for u in range(n):
            flow.add_edge(s, u, 1)
        for v in range(m):
            flow.add_edge(n + v, t, 1)
        for u, v in edges:
            flow.add_edge(u, n + v, inf)
        flow.add_edge(s, forced, inf)
        answer = min(answer, flow.max_flow(s, t) - n)
        if answer < 0:
            break
    print(-1 if answer < 0 else answer)


if __name__ == '__main__':
    main()
```

15 N. Signal Concentration

15.1 Restatement and why total reward is insufficient

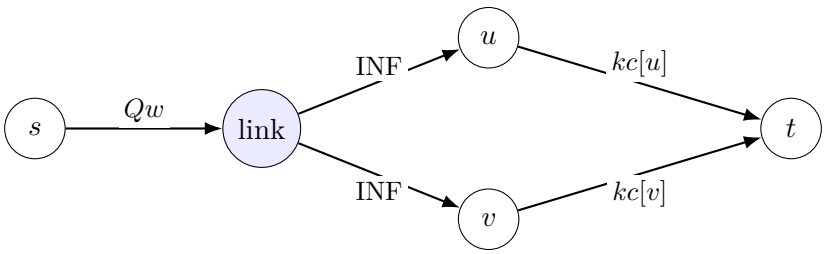
We maximize internal link reward divided by chosen station weight. A set with greater total reward may have a worse ratio. The empty set has no defined ratio and is forbidden, while isolated stations can be omitted. Let `W=sum c[i]` and `R=sum link rewards`. If `R=0`, every nonempty choice has ratio zero; print `0 1`.

15.2 Turn a ratio threshold into an additive objective

Choose integer scale `Q=W^2+1`. For an integer price `k`, ask whether some station set has

```
Q * internal_reward - k * chosen_weight > 0.
```

This means its ratio strictly exceeds `k/Q`. Build an auxiliary node for each link of reward `w`. Its selection claims reward only when both endpoints are selected:



The station-to-sink edges are added **once per station**, not once per incident link. Choose `INF=Q*R+1`. With selected vertices on the source side, a finite cut may claim a link only if both endpoints are selected. For fixed stations it can claim every internal positive link, so

```
maximum additive score = Q*R - minimum_cut_capacity.
```

The empty selection has score zero. Strict positivity therefore certifies a nonempty set without forcing an arbitrary station. An auxiliary node's infinite edges are logical requirements, not routing capacities from the original undirected graph.

15.3 Exact binary search and extraction

The strict predicate is true at `k=0` when `R>0`, and false at `k=Q*R+1`, because any nonempty set has weight at least one and reward at most `R`. Binary-search the largest true integer `k`. Then

`k/Q < optimal_ratio <= (k+1)/Q.`

Run the positive test at that final `k` again and extract its residual source side. Sum original station weights and original rewards with both endpoints selected. Reduce the resulting numerator and denominator by gcd. Do not print `k/Q`: it is a search boundary, not necessarily an actual ratio.

15.4 Why the extracted ratio is exactly optimal

Every actual denominator is at most `W`. Unequal actual ratios differ by at least `1/W^2`, because the cross-multiplied numerator difference is a nonzero integer. But the search interval has width `1/Q < 1/W^2`. Any positive-score extracted set has ratio above `k/Q` and at most the optimum. If it were smaller than the optimum, their separation would be at least `1/W^2`, greater than the entire interval containing them. This is impossible. Thus every positive extracted choice at this price is optimal. Strict `>0` is essential, especially if `Q*optimal_ratio` itself is an integer: the last true price is then one less than that integer.

On the sample, sets `{1,2}` and `{1,2,3}` both have ratio 2. The extracted cut may pick either; the original totals reduce to `2 1` in both cases. For a single link of reward 7 and endpoint weights 2 and 3, extraction instead returns the genuinely fractional answer `7 5`.

15.5 Bounds, implementation notes, and tests

Each network has `V=n+m+2`, `E=n+3m`. There are `O(log(Q*R+2))` flow tests plus one extraction, giving general time `O(log(Q*R+2)*V^2 E)` and `O(n+m)` working memory. With `W<=1200`, `Q<=1440001`, and `R<=3*10^8`, the binary-search upper bound is below `4.321*10^14`; multiplying it by any station weight at most 20 remains below `8.642*10^15`. These values fit signed 64 bits. Python uses exact integers throughout. No floating-point tolerance is involved.

The small oracle enumerates station subsets and compares exact `Fraction` values. Tests include all-zero rewards, isolated stations, noninteger answers, and a large low-density component that must be omitted. A maximum regular graph gives a known degree-based density bound. Another stress case has two 30-station components with ratios `599001/599` and `600001/600`, separated by less than `0.000003`; this checks exact recovery near a tie.

15.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
```

```
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
```

```
            }
            return delta;
        }
        while (ptr[u] < static_cast<int>(adj[u].size())) {
            int edge = adj[u][ptr[u]];
            int v = edges[edge].to;
            if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                break;
            ++ptr[u];
        }
        if (ptr[u] == static_cast<int>(adj[u].size())) {
            vertices.pop_back();
            bottleneck.pop_back();
            if (!path.empty()) {
                path.pop_back();
                ++ptr[vertices.back()];
            }
        } else {
            int edge = adj[u][ptr[u]];
            path.push_back(edge);
            vertices.push_back(edges[edge].to);
            bottleneck.push_back(
                std::min(bottleneck.back(), edges[edge].cap));
        }
    }
    return 0;
}

long long max_flow(int s, int t) {
    assert(0 <= s && s < n && 0 <= t && t < n && s != t);
    long long added = 0, limit = 0;
    for (int edge : adj[s]) limit += edges[edge].cap;
    while (bfs(s, t)) {
        ptr.assign(n, 0);
        while (long long delta = send(s, t, limit)) added += delta;
    }
    return added;
}

std::vector<bool> reachable(int s) const {
    std::vector<bool> seen(n, false);
    seen[s] = true;
    std::queue<int> queue;
    queue.push(s);
    while (!queue.empty()) {
        int u = queue.front();
        queue.pop();
        for (int edge : adj[u]) {
            int v = edges[edge].to;
            if (edges[edge].cap > 0 && !seen[v]) {
                seen[v] = true;
```

```
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<long long> cost(n);
    long long weight = 0;
    for (auto& x : cost) { cin >> x; weight += x; }
    vector<tuple<int, int, long long>> links(m);
    long long reward = 0;
    for (auto& [u, v, w] : links) { cin >> u >> v >> w; --u; --v; reward += w; }
    if (reward == 0) { cout << "0 1\n"; return 0; }
    long long scale = weight * weight + 1;
    long long scaled_reward = scale * reward;
    auto evaluate = [&](long long price, vector<bool>* chosen) {
        int s = n + m, t = s + 1;
        Dinic flow(t + 1);
        for (int v = 0; v < n; ++v) flow.add_edge(v, t, price * cost[v]);
        for (int j = 0; j < m; ++j) {
            auto [u, v, w] = links[j];
            int a = n + j;
            flow.add_edge(s, a, scale * w);
            flow.add_edge(a, u, scaled_reward + 1);
            flow.add_edge(a, v, scaled_reward + 1);
        }
        bool positive = scaled_reward - flow.max_flow(s, t) > 0;
        if (chosen) *chosen = flow.reachable(s);
        return positive;
    };
    long long lo = 0, hi = scaled_reward + 1;
    while (hi - lo > 1) {
        long long mid = lo + (hi - lo) / 2;
        if (evaluate(mid, nullptr)) lo = mid;
        else hi = mid;
    }
    vector<bool> chosen;
    evaluate(lo, &chosen);
    long long numerator = 0, denominator = 0;
    for (int v = 0; v < n; ++v) if (chosen[v]) denominator += cost[v];
    for (auto [u, v, w] : links)
        if (chosen[u] && chosen[v]) numerator += w;
    long long g = gcd(numerator, denominator);
    cout << numerator / g << ' ' << denominator / g << '\n';
```

```
}
```

15.7 Full Python solution

```python
import sys
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
```

```
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
```

```
        return seen

from math import gcd


def main():
    values = list(map(int, sys.stdin.buffer.read().split()))
    if not values:
        return
    it = iter(values)
    n, m = next(it), next(it)
    costs = [next(it) for _ in range(n)]
    links = [(next(it) - 1, next(it) - 1, next(it)) for _ in range(m)]
    reward = sum(w for u, v, w in links)
    if reward == 0:
        print('0 1')
        return
    weight = sum(costs)
    scale = weight * weight + 1
    scaled_reward = scale * reward
    def evaluate(price, extract=False):
        s, t = n + m, n + m + 1
        flow = Dinic(t + 1)
        for v, cost in enumerate(costs):
            flow.add_edge(v, t, price * cost)
        for j, (u, v, w) in enumerate(links):
            a = n + j
            flow.add_edge(s, a, scale * w)
            flow.add_edge(a, u, scaled_reward + 1)
            flow.add_edge(a, v, scaled_reward + 1)
        positive = scaled_reward - flow.max_flow(s, t) > 0
        return flow.reachable(s) if extract else positive
    lo, hi = 0, scaled_reward + 1
    while hi - lo > 1:
        mid = (lo + hi) // 2
        if evaluate(mid):
            lo = mid
        else:
            hi = mid
    chosen = evaluate(lo, True)
    numerator = sum(w for u, v, w in links if chosen[u] and chosen[v])
    denominator = sum(c for v, c in enumerate(costs) if chosen[v])
    g = gcd(numerator, denominator)
    print(numerator // g, denominator // g)


if __name__ == '__main__':
    main()
```

16 Testing and what to carry forward

Every E-N package contains fixed examples, an independent tiny oracle, random cases, and maximum-size stress cases. The oracles enumerate original choices: placements, shipments, assignments, closures, binary decisions, pact selections, lane amounts, actual working days, cancellations, or station subsets. They do not call a reference flow implementation to decide the expected answer.

```
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py --keep-going
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py --problem k,l,m,n
```

Before implementing a new model, write down its interpretation of one unit, a chosen pair, or a cut side. Check that each original feasible choice gives a graph solution and each graph solution gives an original feasible choice. For binary search, prove monotonicity and rebuild the graph between tests. For infinite capacities, exhibit a finite cut cheaper than `INF`. For compressed time, prove that aggregate amounts can be realized on actual days. For exact ratios, bound the denominator and every integer product. These proofs catch modeling mistakes that a correct Dinic routine cannot fix.